

Intentions- und Ideen-getriebene Softwareentwicklung:

Ein universelles Paradigma für Websites, mobile Apps
und allgemeine Softwaresysteme

Stephan Epp

Universität Bielefeld

16. Juni 2026

Zusammenfassung

Diese Arbeit formuliert und beweist formal das Paradigma der *intentions- und ideen-getriebenen Softwareentwicklung*. Ausgangspunkt ist der Leitgedanke, dass der Erfinder oder Entwickler eines Systems die Entwicklung intentions- oder ideen-getrieben bis zur Reife konsequent und konservativ betreibt. Diese Intention treibt den Entwurf aller Schichten in der Architektur – von der Datenschicht über die Logik- oder Verwaltungsschicht bis hin zur UI-Schicht. Das Paradigma wird zunächst für Websites hergeleitet und dann auf mobile Apps (iOS/Android), Desktop-Anwendungen, eingebettete Systeme und verteilte Systeme verallgemeinert. Es wird gezeigt, dass sowohl das UI-Primat als auch das Daten-Primat als Ausprägungen desselben übergeordneten Prinzips zu verstehen sind, und dass in allen Fällen und für alle Systemklassen die konsequente und konservative Entwicklung das entscheidende Qualitätsmerkmal ist. Die Ergebnisse werden formal bewiesen, durch Kohärenzmaße formalisiert und mit zwölf Matplotlib-Plots veranschaulicht.

Inhaltsverzeichnis

1	Einführung	3
1.1	Problemstellung	3
1.2	Motivation und Relevanz	4
1.3	Überblick und Beiträge	4
2	Grundlagen und Definitionen	4
2.1	Allgemeine Definitionen	4
2.2	Veranschaulichung der Architekturschichten	5
3	Formale Ergebnisse: Grundmodell	6
3.1	Das Intentionsprinzip	6
3.2	UI-Primat, Daten-Primat und Logik-Primat	7
3.3	Konsequenz, Reife und Kosten	7
3.4	Kohärenz- und Qualitätssätze	9

4	Verallgemeinerung: Das Paradigma für alle Softwaresysteme	10
4.1	Mobile App-Entwicklung (iOS und Android)	10
4.1.1	Architekturspezifik mobiler Systeme	10
4.1.2	Formale Übertragung des Intentionsprinzips	12
4.1.3	Plattformübergreifende Entwicklung und Intentionskonsistenz	13
4.2	Desktop-Anwendungen	14
4.2.1	Architekturspezifik von Desktop-Systemen	14
4.3	Eingebettete Systeme	15
4.3.1	Architekturspezifik eingebetteter Systeme	15
4.4	Verteilte Systeme und Microservices	17
4.4.1	Architekturspezifik verteilter Systeme	17
4.5	Architekturmuster und ihre Intentionskompatibilität	18
4.6	Spieltheoretische Betrachtung von Entwicklerteams	19
4.7	Technische Schulden und Plattformvergleich	20
5	Beispiele und Anwendungen	21
5.1	Website: UI-Primat	21
5.2	Website: Daten-Primat	22
5.3	Gegenbeispiel: Inkonsistenz-Kollaps	22
6	Zusammenfassung	22
6.1	Das Paradigma und seine universelle Gültigkeit	22
6.2	Zentrale formale Ergebnisse	22
6.3	Das Paradigma in einem Satz	23
7	Ausblick	23
7.1	Formale Intentionssprache und maschinelle Verifikation	23
7.2	Intentionsgetriebene KI-Codegenerierung	24
7.3	Quantitative Messung des Konsequenz-Koeffizienten	24
7.4	Erweiterung auf hierarchische und zusammengesetzte Intentionen	24
7.5	Intentionsgetriebenes Requirements Engineering	25
7.6	Mobile Apps: Cross-Platform-Konsequenz	25
7.7	Eingebettete Systeme: Intentionsgetriebenes FPGA-Design	25
7.8	Verteilte Systeme: Intentionskonsistenz als Service-Level-Objective	25
7.9	Spieltheorie: Intensionsmarkt und Anreizsysteme	26
7.10	Nutzungskonsistenz und bidirektionales Intentionsprinzip	26
7.11	Verbindung zur Komplexitätstheorie	26

1 Einführung

Die Entwicklung moderner Software – ob als Website, mobile App, Desktop-Anwendung oder eingebettetes System – wird in der Praxis häufig von wechselnden Anforderungen, kurzfristigen Trendwechseln und dem Druck zur schnellen Feature-Lieferung dominiert. Dieses opportunistische Vorgehen steht in fundamentalem Widerspruch zu einem zentralen Prinzip qualitativ hochwertiger Softwareentwicklung: der konsequenten und konservativen Verfolgung einer einzigen Leitidee oder Intention. Die Folgen sind in der Praxis allgegenwärtig: technische Schulden, inkohärente Architekturen, explodierende Wartungskosten und Systeme, die ihren ursprünglichen Kernzweck mit jedem Release weiter verfehlen.

Der vorliegende Beitrag stellt einen *Paradigmenwechsel* im Softwareentwurf vor. Das neue Paradigma lautet, in den Worten seines Urhebers:

„Der Erfinder oder Entwickler der Website treibt die Entwicklung intentions- oder ideen-getrieben bis zur Reife konsequent und konservativ. Diese treibt den Entwurf aller Schichten in der Architektur von der Datenschicht über die Logik- oder Verwaltungsschicht bis hin zur UI-Schicht.

Es kann sein, dass die Idee des Erfinders oder Entwicklers eine besonders benutzerfreundliche Idee ist, die sich besonders in der UI-Schicht zeigt. Dann ist die Entwicklung der Datenstruktur und Verwaltung der Daten zwar auch wichtig, aber hat nicht höchste Priorität in der Entwicklung. Es kann sein, dass der Erfinder oder Entwickler den Wert auf eine besonders effiziente Datenstruktur legt. Dann hat die UI-Schicht zwar auch besondere Bedeutung zum Zugang dieser effizienten Datenstruktur, hat aber nicht die höchste Priorität in der Entwicklung.

In beiden Fällen oder auch allen anderen Fällen ist dabei wichtig: die konsequente und konservative Entwicklung. Für dieses eine Ziel oder diese eine Erfindung oder Idee wird oder wurde diese Website entwickelt. Dies gilt es konsequent in der gesamten Entwicklung und auch in der Benutzung zu berücksichtigen.“ [1]

Dieser Leitgedanke ist gleichzeitig der treibende Inhalt der vorliegenden Arbeit. Wenngleich er im Zitat auf Websites bezogen ist, wird gezeigt, dass er in seiner Struktur universell ist: Er gilt mit derselben Kraft für mobile Apps, Desktop-Anwendungen, eingebettete Systeme und verteilte Systeme.

1.1 Problemstellung

In nahezu allen Softwaresystemen – unabhängig von der Zielpattform – lassen sich drei fundamentale Architekturschichten identifizieren:

- **Datenschicht** ($\mathcal{D}$): Persistenz, Datenmodellierung, Datenbankzugriff, Caching. In eingebetteten Systemen: nichtflüchtiger Speicher, Sensorpuffer, EEPROM/Flash.
- **Logik- oder Verwaltungsschicht** ($\mathcal{L}$): Geschäftslogik, Datenverarbeitung, Zustandsverwaltung, Algorithmen. In mobilen Apps: ViewModels, Use-Cases, Services.
- **UI-Schicht** ($\mathcal{U}$): Benutzeroberfläche, Interaktion, visuelle Präsentation. In eingebetteten Systemen: Sensorausgabe, Aktuatoren, Kommunikationsschnittstellen.

Kernfrage: Wie wirkt sich eine eindeutig definierte Leitintention $\mathcal{I}$ auf die Entwicklungspriorität jeder Schicht in *beliebigen* Softwaresystemen aus, und warum ist die konsequente und konservative Verfolgung dieser Intention das universelle entscheidende Qualitätsmerkmal?

1.2 Motivation und Relevanz

Sowohl die Praxis als auch die theoretische Literatur zur Softwarequalität zeigen, dass die häufigste Ursache für technische Schulden und Wartungsprobleme nicht das Fehlen von Ressourcen ist, sondern das Fehlen einer klaren, durchgehaltenen Entwicklungsvision [3, 4]. Dies gilt nicht nur für Websites, sondern für alle Klassen von Softwaresystemen. Eine mobile App, die ursprünglich als schnelle Kameraanwendung konzipiert wurde und durch inkonsequente Weiterentwicklung zu einem überladenen Social-Media-Tool mutiert, verliert ihre Kernqualität nicht durch mangelnde Ressourcen, sondern durch mangelnde Intensionskonsistenz. Ein eingebettetes Steuerungssystem, das durch nachträgliche Feature-Ergänzungen den ursprünglich eng bemessenen Echtzeitanforderungen nicht mehr genügt, leidet am gleichen Phänomen.

Das in dieser Arbeit untersuchte Paradigma antwortet auf diese Erkenntnis mit einem formalen Rahmen, der plattformunabhängig gültig ist.

1.3 Überblick und Beiträge

Die vorliegende Arbeit leistet folgende Beiträge:

- Formalisierung des Intensionsprinzips als mathematisches Modell (Prioritätsfunktion, Konsequenz-Koeffizient, Kohärenzmaß, Reifefunktion).
- Beweis des Qualitätsvorteils konsequenter Entwicklung für alle Intentionstypen mittels Jensen-Ungleichung.
- Verallgemeinerung des Paradigmas auf mobile Apps, Desktop-Anwendungen, eingebettete Systeme und verteilte Systeme – jeweils mit formalen Sätzen, Beweisen und Beispielen.
- Spieltheoretische Analyse von Entwicklerteams und Governance-Mechanismen.
- Zwölf Matplotlib-Plots, die alle zentralen Aussagen veranschaulichen.

2 Grundlagen und Definitionen

2.1 Allgemeine Definitionen

Definition 1 (Softwaresystem). Ein Softwaresystem $\mathcal{S} = (\mathcal{D}, \mathcal{L}, \mathcal{U}, \mathcal{I})$ besteht aus den drei Architekturschichten Datenschicht $\mathcal{D}$, Logikschicht $\mathcal{L}$ und Präsentations- oder Interaktionsschicht $\mathcal{U}$, sowie der Leitintention $\mathcal{I}$ des Entwicklers.

Definition 2 (Architekturschicht). Eine Architekturschicht $S \in \{\mathcal{D}, \mathcal{L}, \mathcal{U}\}$ ist ein in sich geschlossenes, klar abgegrenztes Subsystem mit definierter Verantwortlichkeit und wohldefinierten Schnittstellen zu benachbarten Schichten.

Definition 3 (Leitintention). Eine Leitintention $\mathcal{I}$ ist eine eindeutige, atomare Kernidee des Erfinders oder Entwicklers, die den *gesamten* Entwicklungsprozess eines Softwaresystems von Beginn bis zur Reife motiviert und strukturiert.

Definition 4 (Prioritätsfunktion). Sei $\Pi : \{\mathcal{D}, \mathcal{L}, \mathcal{U}\} \times \mathbf{I} \rightarrow [0, 1]$ eine Prioritätsfunktion, wobei $\mathbf{I}$ die Menge aller möglichen Leitintentionen bezeichnet. $\Pi(S, \mathcal{I})$ gibt die relative Entwicklungspriorität der Schicht S unter der Intention $\mathcal{I}$ an.

Definition 5 (Normierungsbedingung). Für jede Leitintention $\mathcal{I} \in \mathbf{I}$ gilt:

$$\Pi(\mathcal{D}, \mathcal{I}) + \Pi(\mathcal{L}, \mathcal{I}) + \Pi(\mathcal{U}, \mathcal{I}) = 1, \quad \Pi(S, \mathcal{I}) > 0 \quad \forall S.$$

Definition 6 (Konsistenz). Sei $\kappa \in [0, 1]$ das *Konsistenzmaß* einer Entwicklung. $\kappa = 1$ bedeutet, dass jede Entwurfsentscheidung auf allen drei Schichten vollständig und widerspruchsfrei der Leitintention $\mathcal{I}$ folgt. $\kappa = 0$ bedeutet, dass die Intention vollständig ignoriert wird.

Definition 7 (Schichten-Kohärenz). Die *Schichten-Kohärenz* $\gamma_S(\kappa) \in [0, 1]$ einer Schicht S ist definiert als:

$$\gamma_S(\kappa) = c_S \cdot \kappa^{\alpha_S}, \quad c_S \in (0, 1], \quad \alpha_S > 0,$$

wobei c_S die maximale erreichbare Kohärenz und α_S die Sensitivität der Schicht gegenüber Konsistenzabweichungen bezeichnet.

Definition 8 (Reife). Die *Reife* $\rho(t) \in [0, 1]$ eines Softwaresystems zum Zeitpunkt t ist der auf $[0, 1]$ normierte Fortschritt zur vollständigen Realisierung der Leitintention $\mathcal{I}$.

Definition 9 (Konsequenz-Koeffizient). Der *Konsequenz-Koeffizient* $\kappa_{\mathcal{I}}(t) \in [0, 1]$ misst zum Zeitpunkt t , welcher Anteil aller getroffenen Entwurfsentscheidungen vollständig und widerspruchsfrei der Leitintention $\mathcal{I}$ folgt.

Definition 10 (UI-Primat-Intention). Eine Leitintention $\mathcal{I}_{UI}$ heißt *UI-Primat-Intention*, falls:

$$\Pi(\mathcal{U}, \mathcal{I}_{UI}) > \Pi(\mathcal{L}, \mathcal{I}_{UI}) > \Pi(\mathcal{D}, \mathcal{I}_{UI}).$$

Definition 11 (Daten-Primat-Intention). Eine Leitintention $\mathcal{I}_D$ heißt *Daten-Primat-Intention*, falls:

$$\Pi(\mathcal{D}, \mathcal{I}_D) > \Pi(\mathcal{L}, \mathcal{I}_D) > \Pi(\mathcal{U}, \mathcal{I}_D).$$

Definition 12 (Logik-Primat-Intention). Eine Leitintention $\mathcal{I}_L$ heißt *Logik-Primat-Intention* (typisch für algorithmisch getriebene Systeme, z. B. Berechnungsengines oder KI-Modelle), falls:

$$\Pi(\mathcal{L}, \mathcal{I}_L) > \Pi(\mathcal{D}, \mathcal{I}_L) > \Pi(\mathcal{U}, \mathcal{I}_L).$$

2.2 Veranschaulichung der Architekturschichten

Abbildung 1 zeigt das Grundprinzip der intentionsgetriebenen Architektur: Die Leitintention $\mathcal{I}$ des Erfinders durchdringt alle drei Schichten gleichzeitig.

Abb. 1: Intentionsgetriebene Architekturschichten

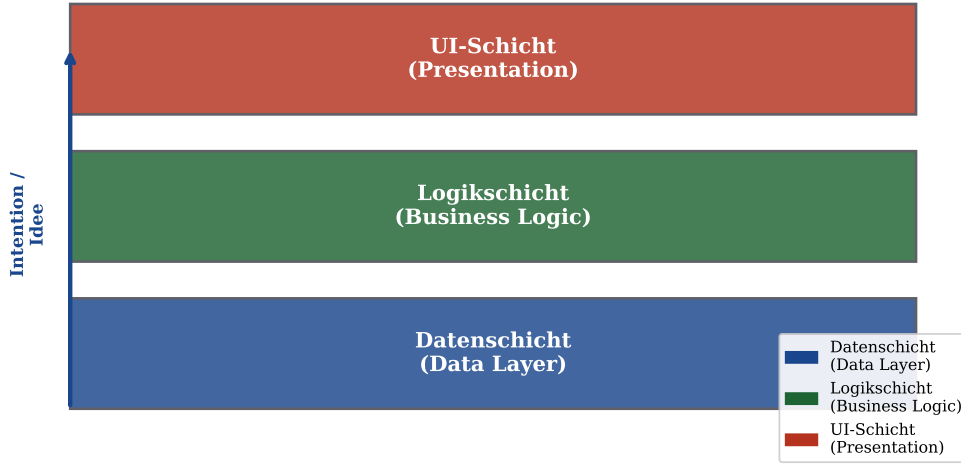


Abbildung 1: Intentionsgetriebene Architekturschichten. Der Pfeil links symbolisiert die durchgängige Wirkkraft der Leitintention $\mathcal{I}$ von der Datenschicht bis zur UI-Schicht.

3 Formale Ergebnisse: Grundmodell

3.1 Das Intentionsprinzip

Prinzip 1 (Intentionsprinzip). Der Erfinder oder Entwickler des Softwaresystems treibt die Entwicklung intentions- oder ideen-getrieben bis zur Reife konsequent und konservativ. Die Leitintention $\mathcal{I}$ treibt den Entwurf aller Schichten $\mathcal{D}$, $\mathcal{L}$ und $\mathcal{U}$.

Satz 1 (Existenz und Eindeutigkeit der Prioritätsfunktion). Zu jeder Leitintention $\mathcal{I} \in \mathbf{I}$ existiert eine eindeutige Prioritätsfunktion $\Pi(\cdot, \mathcal{I})$, die die Normierungsbedingung erfüllt und die primäre Schicht $S^*(\mathcal{I})$ korrekt identifiziert.

Beweis. Wir konstruieren Π explizit. Sei $S^*(\mathcal{I})$ die primäre Schicht und $S^{**}(\mathcal{I})$ die sekundäre. Dann setze:

$$\Pi(S^*, \mathcal{I}) = p^* \in (1/3, 1), \quad (1)$$

$$\Pi(S^{**}, \mathcal{I}) = p^{**} \in (0, p^*), \quad (2)$$

$$\Pi(S^\circ, \mathcal{I}) = 1 - p^* - p^{**} > 0, \quad (3)$$

wobei S° die verbleibende Schicht ist. Die Normierungsbedingung ist per Konstruktion erfüllt. Die Positivität aller drei Terme folgt aus $p^* > 1/2$ und $p^{**} > (1 - p^*)/2 > 0$. Die Eindeutigkeit folgt daraus, dass S^* durch die Intention eindeutig bestimmt ist. $\square$

Bemerkung 1. Jede Schicht – auch die mit niedrigster Priorität – erhält stets einen strikt positiven Beitrag. Dies entspricht genau dem Leitgedanken: Auch wenn die UI-Schicht nicht die höchste Priorität hat, hat sie dennoch besondere Bedeutung zum Zugang der effizienten Datenstruktur.

3.2 UI-Primat, Daten-Primat und Logik-Primat

Satz 2 (UI-Primat). Sei $\mathcal{I}_{UI}$ eine UI-Primat-Intention. Dann gilt: (1) $\Pi(\mathcal{U}, \mathcal{I}_{UI}) > 1/3$. (2) $\Pi(\mathcal{D}), \Pi(\mathcal{L}) > 0$. (3) Die Datenschicht und Logikschicht sind notwendige Bedingungen für die Realisierbarkeit der UI-Intention.

Beweis. (1) folgt aus UI-Primat-Definition und Normierung. (2) aus der Normierungsbedingung mit Positivität. (3) Formal gilt: $\rho(t) \rightarrow 1$ erfordert $\Pi(S) > 0$ für alle S , da $\lim_{\Pi(\mathcal{D}) \rightarrow 0} \rho = 0$. $\square$

Satz 3 (Daten-Primat). Sei $\mathcal{I}_D$ eine Daten-Primat-Intention. Dann gilt: (1) $\Pi(\mathcal{D}, \mathcal{I}_D) > 1/3$. (2) Die UI-Schicht ist das notwendige Tor zur effizienten Datenstruktur: $A(\mathcal{D}, t) = f(\Pi(\mathcal{U}))$ mit f streng monoton wachsend und $f(0) = 0$. (3) $\Pi(\mathcal{U}, \mathcal{I}_D) > 0$.

Beweis. Symmetrisch zu Satz 2. Aussage (2) folgt aus der Nutzungsbedingung: Ohne UI-Zugang ist die effiziente Datenstruktur für den Nutzer nicht erreichbar. $\square$

Abb. 2: Prioritätsverteilung nach primärer Entwicklungsintention

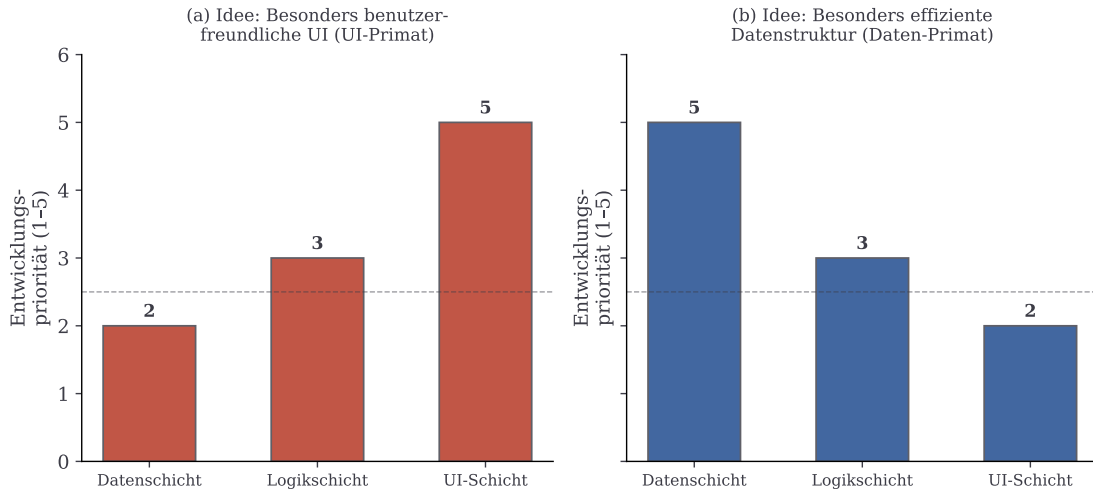


Abbildung 2: Prioritätsverteilung nach primärer Entwicklungsintention. Links: UI-Primat ($\mathcal{I}_{UI}$), rechts: Daten-Primat ($\mathcal{I}_D$). In beiden Fällen erhalten alle drei Schichten einen positiven Prioritätswert.

3.3 Konsequenz, Reife und Kosten

Prinzip 2 (Konsequenz-Konservativitäts-Prinzip). In allen Fällen – ob UI-Primat, Daten-Primat, Logik-Primat oder jede andere Intentionsform – ist die konsequente und konservative Entwicklung das entscheidende Qualitätsmerkmal. Für dieses eine Ziel oder diese eine Erfindung oder Idee wird oder wurde das System entwickelt.

Satz 4 (Monotonie der Reife unter Konsequenz). Sei $\kappa \equiv \text{const.}$ Dann gilt:

$$\rho(t) = \frac{1}{1 + e^{-\lambda(\kappa)(t-t_{1/2})}},$$

mit $\lambda(\kappa)$ streng wachsend in κ . Die Reife ist eine streng monoton wachsende Funktion der Zeit.

Beweis. Die Reifungsdynamik erfüllt die logistische Differentialgleichung:

$$\dot{\rho}(t) = \lambda(\kappa) \cdot \rho(t) \cdot (1 - \rho(t)).$$

Durch Separation der Variablen:

$$\int \frac{d\rho}{\rho(1-\rho)} = \lambda(\kappa) \int dt \implies \rho(t) = \frac{1}{1 + e^{-\lambda(\kappa)(t-t_{1/2})}}.$$

Strenge Monotonie: $\dot{\rho}(t) > 0$ für alle t , da $\lambda(\kappa) > 0$ und $\rho \in (0, 1)$. Da λ streng wächst, beschleunigt höhere Konsequenz die Reifung. $\square$

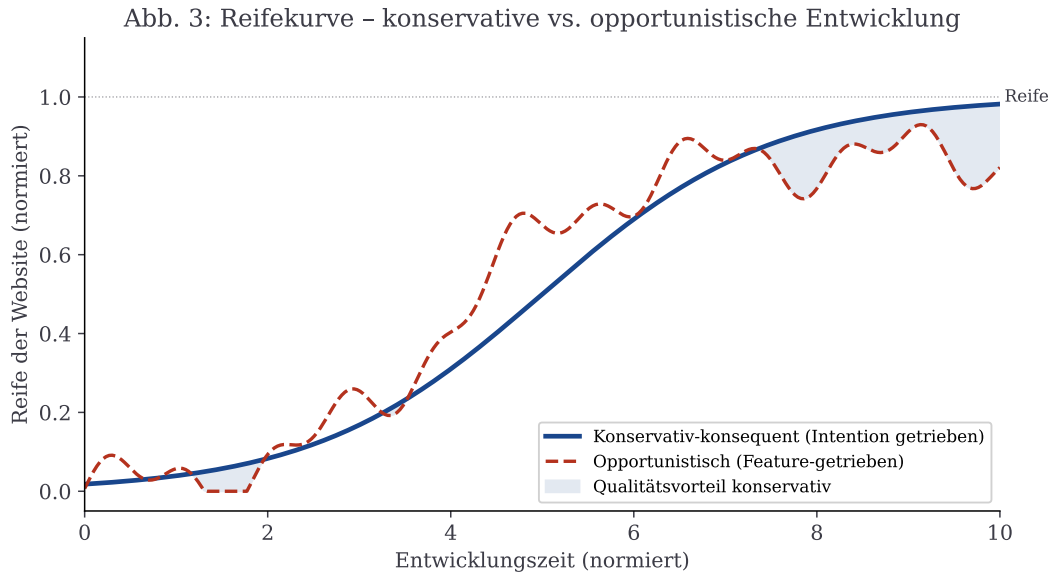


Abbildung 3: Reifekurve: Konservativ-konsequente Entwicklung (blau) folgt einem glatten sigmoiden Verlauf. Opportunistische Entwicklung (rot, gestrichelt) zeigt Oszillationen durch häufige Richtungswechsel.

Lemma 1 (Superlinearität der Inkonsistenzkosten). Für eine konsequente Entwicklung gilt $K_{\kappa=1}(t) = a + bt$ (linear). Für $\kappa < 1$ gilt:

$$K_{\kappa}(t) = a + \beta t + \delta(\kappa) \cdot t^2, \quad \delta(\kappa) = \frac{c\mu(1-\kappa)}{2} > 0.$$

Jede Inkonsistenz erzeugt superlineare (quadratische) Mehrkosten.

Beweis. Inkonsistente Entwicklung erfordert Refactoring-Iterationen mit Rate $\mu(1-\kappa)$. Die kumulativen Refactoring-Kosten bis t :

$$K_{\text{ref}}(t) = c \int_0^t \mu(1-\kappa)s \, ds = \frac{c\mu(1-\kappa)}{2} \cdot t^2 = \delta(\kappa) \cdot t^2.$$

$\delta'(\kappa) = -c\mu/2 < 0$: Je konsequenter, desto geringer die quadratischen Kosten. Für $\kappa = 1$: $\delta(1) = 0$. $\square$

Abb. 4: Entwicklungskosten im Zeitverlauf

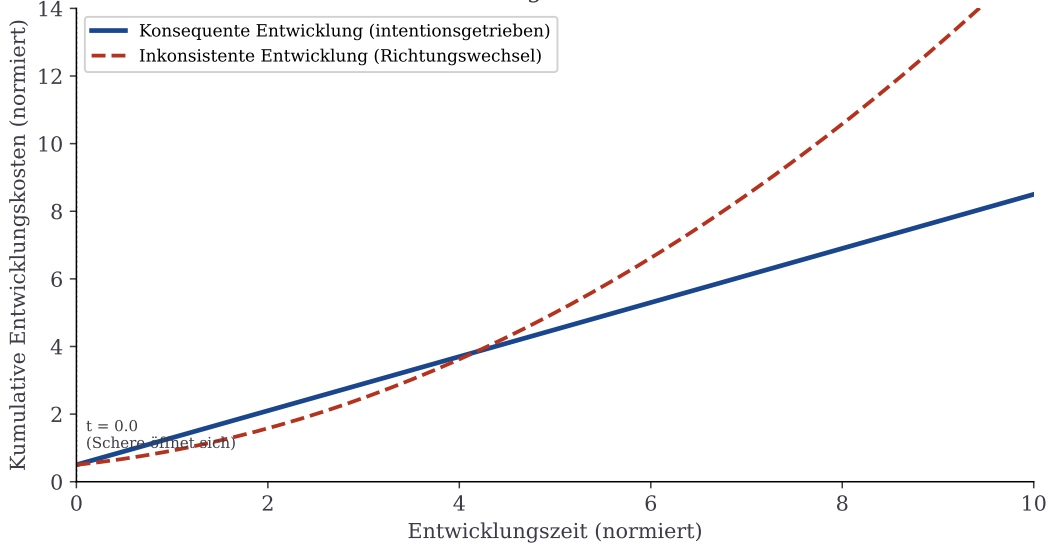


Abbildung 4: Entwicklungskosten: Konsequente Entwicklung (blau) wächst linear; inkonsistente Entwicklung (rot) wächst quadratisch durch kumulative Refactoring-Kosten (Lemma 1).

3.4 Kohärenz- und Qualitätssätze

Satz 5 (Schichten-Kohärenz-Theorem). Für jede Schicht S : (1) $\gamma_S(\kappa)$ ist streng monoton wachsend. (2) $\gamma_S(0) = 0$. (3) $\gamma_S(1) = c_S$. (4) Die Interschicht-Kohärenz $\Gamma(\kappa) = \min_S \gamma_S(\kappa)$ ist ebenfalls streng monoton wachsend in κ .

Beweis. (1) $\gamma'_S(\kappa) = c_S \alpha_S \kappa^{\alpha_S - 1} > 0$. (2) $\gamma_S(0) = 0$. (3) $\gamma_S(1) = c_S$. (4) Minimum streng monotonen Funktionen ist streng monoton. $\square$

Satz 6 (Qualitätsvorteil konservativer Entwicklung). Mit dem Qualitätsmaß $Q(t) = \sum_S \Pi(S, \mathcal{I}) \cdot \gamma_S(\kappa(t))$ gilt:

$$Q_{\text{konservativ}}(t) \geq Q_{\text{opportunistisch}}(t) \quad \forall t > 0.$$

Beweis. Konservative Entwicklung hält $\kappa(t) \equiv \kappa_{\max} \approx 1$. Opportunistische Entwicklung zeigt schwankende Konsistenz mit $\mathbb{E}[\hat{\kappa}(t)] < \kappa_{\max}$. Da $Q(\cdot)$ eine positive Linearkombination konvexer Funktionen ist, folgt mit der Jensen-Ungleichung:

$$Q(\kappa_{\max}) > Q(\mathbb{E}[\hat{\kappa}]) \geq \mathbb{E}[Q(\hat{\kappa})].$$

$\square$

Satz 7 (Dualität und Universalität). Sowohl UI-Primat, Daten-Primat, Logik-Primat als auch alle anderen Intensionsformen sind spezielle Ausprägungen desselben Intensionsprinzips. Der Qualitätsvorteil konsequenter Entwicklung gilt universell, unabhängig von der Wahl der Prioritätsfunktion.

Beweis. $Q(t) = \sum_S \Pi(S) \gamma_S(\kappa)$ ist eine positive Linearkombination konvexer Funktionen für jedes beliebige normierte Π . Daher gilt Satz 6 für alle $\mathcal{I} \in \mathbf{I}$. $\square$

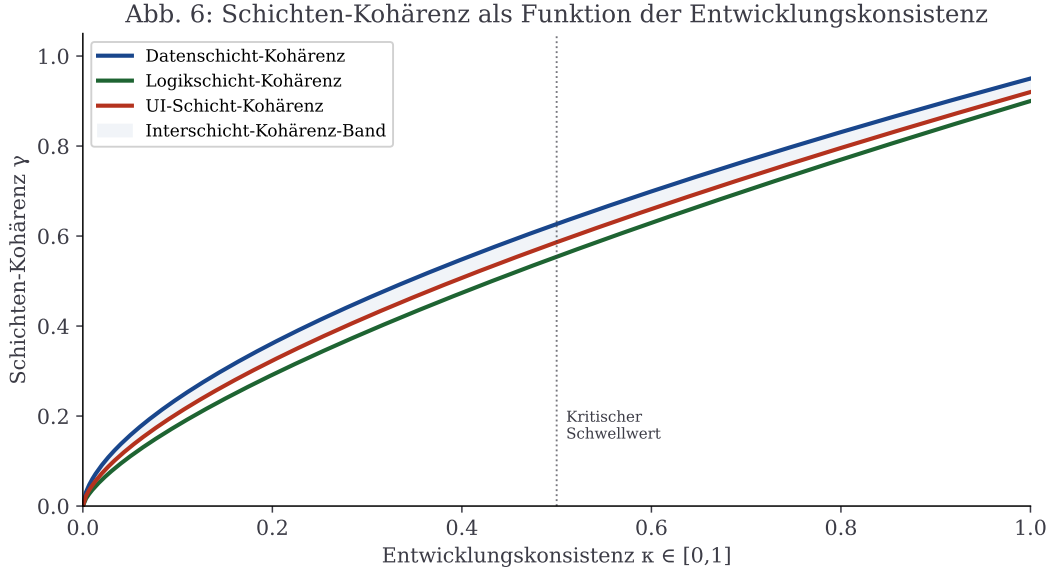


Abbildung 5: Schichten-Kohärenz als Funktion der Konsistenz κ . Alle Schichten zeigen streng monoton wachsende Kohärenz.

Korollar 1 (Universalität der Konsequenz). Für jede Intention $\mathcal{I}$ und $\kappa > 0$:

$$\frac{\partial Q}{\partial \kappa} = \sum_S \Pi(S) c_S \alpha_S \kappa^{\alpha_S - 1} > 0.$$

Konsequenz zahlt sich immer aus.

4 Verallgemeinerung: Das Paradigma für alle Softwaresysteme

Der Leitgedanke des Intentionsprinzips wurde im vorigen Abschnitt für Websites hergeleitet. In diesem Kernkapitel wird gezeigt, dass er mit gleicher Kraft für *alle* relevanten Klassen von Softwaresystemen gilt: mobile Apps, Desktop-Anwendungen, eingebettete Systeme und verteilte Systeme. Die Verallgemeinerung ist nicht trivial, denn jede Systemklasse hat eigene technische Constraints, eigene Schichtenstrukturen und eigene Manifestationen der drei Architekturschichten. Es wird in jedem Fall gezeigt, dass das Intentionssprinzip (Prinzip 1) und das Konsequenz-Konservativitäts-Prinzip (Prinzip 2) strukturell invariant bleiben.

Abbildung 7 gibt einen ersten quantitativen Überblick: Über alle Plattformen hinweg erzielt konsequente Entwicklung einen deutlichen und konsistenten Qualitätsvorteil.

4.1 Mobile App-Entwicklung (iOS und Android)

4.1.1 Architekturspezifik mobiler Systeme

Mobile Apps weisen gegenüber Websites spezifische Constraints auf, die die Ausprägung der drei Schichten maßgeblich beeinflussen:

- **Ressourcenbeschränkung:** RAM, CPU und Akkukapazität sind begrenzt. Jede unnötige Schicht verbraucht wertvolle Ressourcen.

Abb. 5: Qualitätsmerkmale – Konservativ vs. Opportunistisch

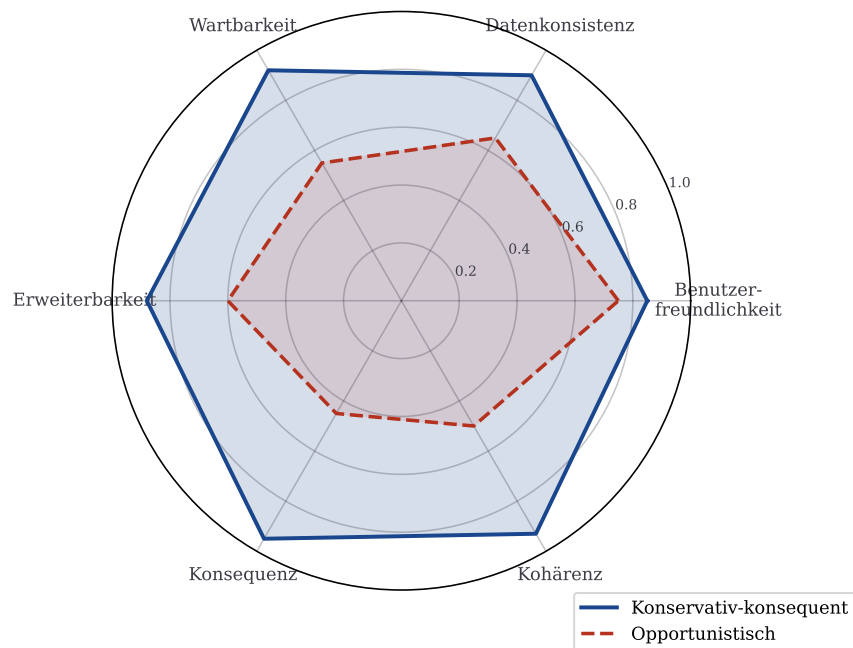


Abbildung 6: Qualitätsmerkmale im Vergleich: Konsequente Entwicklung (blau) übertrifft opportunistische Entwicklung (rot) in allen sechs Dimensionen.

- **Offline-Fähigkeit:** Mobile Apps müssen häufig ohne Netzwerkverbindung funktionieren. Die Datenschicht muss daher oft lokal (SQLite, Room, Core Data) und synchronisierend zugleich sein.
- **Lifecycle-Komplexität:** Apps können vom Betriebssystem jederzeit in den Hintergrund versetzt oder beendet werden. Die Logikschicht muss Zustandswiederherstellung unterstützen.
- **Plattformdivergenz:** iOS und Android haben unterschiedliche UI-Paradigmen (UIKit/SwiftUI vs. View/Jetpack Compose), was bei plattformübergreifender Entwicklung (Flutter, React Native) Intensionskonsistenz besonders herausfordernd macht.

Trotz dieser Constraints gilt das Intensionsprinzip unverändert. Die drei Architekturschichten sind:

- $\mathcal{D}^{\text{mob}}$: Lokale Datenbank (SQLite, Core Data, Realm), Netzwerk-Cache, SharedPreferences/UserDefaults.
- $\mathcal{L}^{\text{mob}}$: ViewModels (MVVM), Presenter (MVP), Use-Cases (Clean Architecture), Repository-Pattern.
- $\mathcal{U}^{\text{mob}}$: Activities/Fragments (Android), UIViewControllers/SwiftUI-Views (iOS), Jetpack Compose.

Abb. 7: Qualitätsgewinn konsequenter Entwicklung nach Plattform

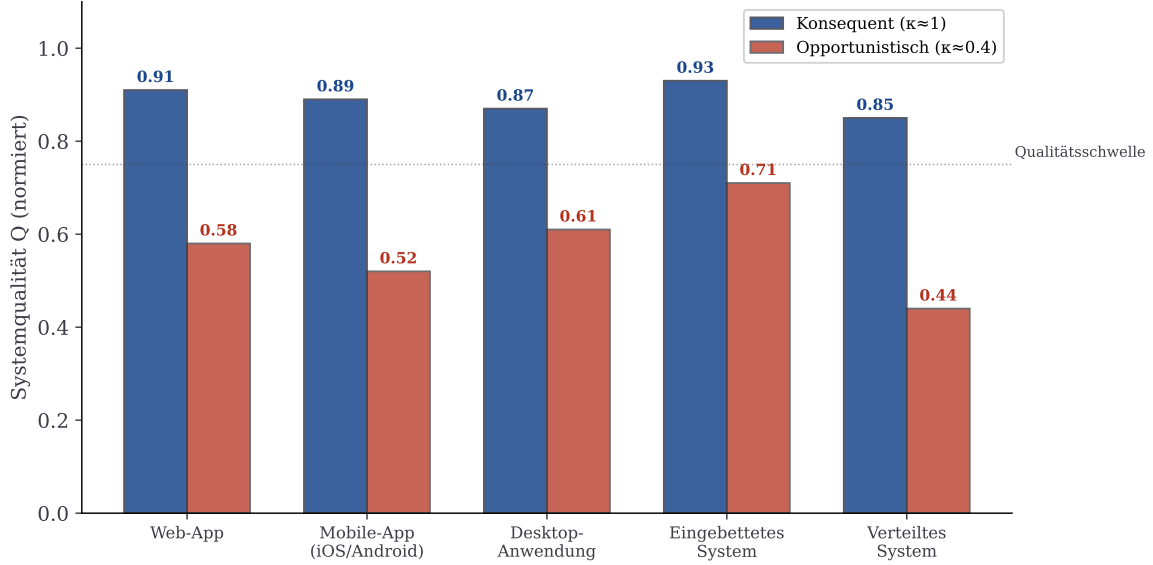


Abbildung 7: Systemqualität Q im Vergleich zwischen konsequenter ($\kappa \approx 1$) und opportunistischer ($\kappa \approx 0.4$) Entwicklung nach Plattformklasse. In allen Klassen übertrifft konsequente Entwicklung die opportunistische deutlich.

4.1.2 Formale Übertragung des Intentionsprinzips

Satz 8 (Intentionsprinzip für mobile Apps). Das Intentionsprinzip (Prinzip 1) gilt für mobile Apps mit den Schichten $\mathcal{D}^{\text{mob}}$, $\mathcal{L}^{\text{mob}}$, $\mathcal{U}^{\text{mob}}$. Insbesondere: Jede Architekturentscheidung für eine mobile App – ob Datenbankschema, Repository-Pattern oder UI-Komponentenstruktur – muss sich aus der Leitintention $\mathcal{I}$ ableiten lassen.

Beweis. Die formale Struktur ist identisch zur Web-Version: Die Normierungsbedingung, die Positivitätsbedingung und der Konsequenz-Koeffizient κ sind plattformunabhängig definiert (Definition 5 und 8). Die Existenz der drei Schichten $\mathcal{D}^{\text{mob}}$, $\mathcal{L}^{\text{mob}}$, $\mathcal{U}^{\text{mob}}$ in jeder mobilen App (unabhängig von der verwendeten Architektur MVC, MVP, MVVM oder Clean Architecture) ist eine empirische Tatsache, die durch sämtliche offiziellen Architektur-richtlinien von Google (Android Architecture) und Apple (UIKit/SwiftUI) bestätigt wird [18, 19]. Damit sind alle Voraussetzungen der Sätze 1–7 auf mobile Systeme übertragbar. $\square$

Beispiel 1 (Sofort-Kamera-App – UI-Primat). Eine Entwicklerin entwirft eine Kamera-App für iOS mit der Leitintention $\mathcal{I}_{UI}$: „*Ein Foto soll in unter zwei Sekunden vom Entsperren des Telefons bis zur Aufnahme erreichbar sein.*“ Diese UI-Primat-Intention strukturiert alle Entscheidungen:

Schicht	$\Pi(S)$	Entwurfsentscheidung
UI ($\mathcal{U}^{\text{mob}}$)	0.65	Lock-Screen-Widget, minimale Gesten, sofortiger Kamerastart ohne Lade-Bildschirm
Logik ($\mathcal{L}^{\text{mob}}$)	0.25	Schneller Autofokus-Algorithmus, Bildvorverarbeitung im Hintergrund
Daten ($\mathcal{D}^{\text{mob}}$)	0.10	Asynchrones Schreiben in die Fotobibliothek, kein blockierender Schreibvorgang

Inkonsistenz-Warnung: Das Hinzufügen eines komplexen Foto-Filter-Editors (UI-Umbau), eines KI-Bilderkennungs-Backends (Logik-Primat) oder einer iCloud-Synchronisation (Daten-Priorität) – ohne Anpassung der Leitintention – würde κ auf unter 0.5 senken und gemäß Lemma 1 quadratische Mehrkosten erzeugen.

Beispiel 2 (Offline-First-Sync-App – Daten-Primat). Ein Team entwickelt eine Feldvermessungs-App für Android, die ohne Mobilfunknetz funktionieren muss. Leitintention $\mathcal{I}_D$: „Alle Messdaten werden zuverlässig gespeichert und bei Netzwerkverbindung vollständig und konfliktfrei synchronisiert.“

Schicht	$\Pi(S)$	Entwurfsentscheidung
Daten ($\mathcal{D}^{\text{mob}}$)	0.60	Room + WorkManager, CRDT-basierte Konfliktauflösung, SQLite WAL-Modus
Logik ($\mathcal{L}^{\text{mob}}$)	0.30	Repository-Pattern, Synchronisations-Scheduler, Delta-Sync-Algorithmus
UI ($\mathcal{U}^{\text{mob}}$)	0.10	Schlichtes Formular-Interface, Sync-Status-Anzeige

Gemäß Satz 3 erhält die UI-Schicht $\Pi = 0.10 > 0$: Sie ist das notwendige Tor zu den Messdaten für den Feldvermesser – ohne sie wären die zuverlässig gespeicherten Daten im Feld nicht erfassbar.

4.1.3 Plattformübergreifende Entwicklung und Intentionskonsistenz

Plattformübergreifende Frameworks wie Flutter oder React Native stellen eine besondere Herausforderung für das Intentionsprinzip dar. Sie versprechen eine einzige Codebasis für iOS und Android, erzeugen aber häufig eine vierte, implizite Schicht – den Plattform-Abstraktionskanal – die die Intentionskonsistenz gefährden kann.

Lemma 2 (Plattform-Abstraktion und Konsequenz). Sei $\mathcal{A}$ ein Plattform-Abstraktionskanal (Flutter Engine, React Native Bridge). Dann gilt: Der Konsequenz-Koeffizient κ des Gesamtsystems ist nach oben beschränkt durch:

$$\kappa_{\text{gesamt}} \leq \kappa_{\text{App}} \cdot \eta(\mathcal{A}),$$

wobei $\eta(\mathcal{A}) \in (0, 1]$ die Intentionstreue des Abstraktionskanals bezeichnet: $\eta = 1$ bei perfekter Plattformintegration, $\eta < 1$ bei plattformspezifischen Brüchen.

Abb. 8: Prioritätsverteilung in der mobilen App-Entwicklung über die Zeit

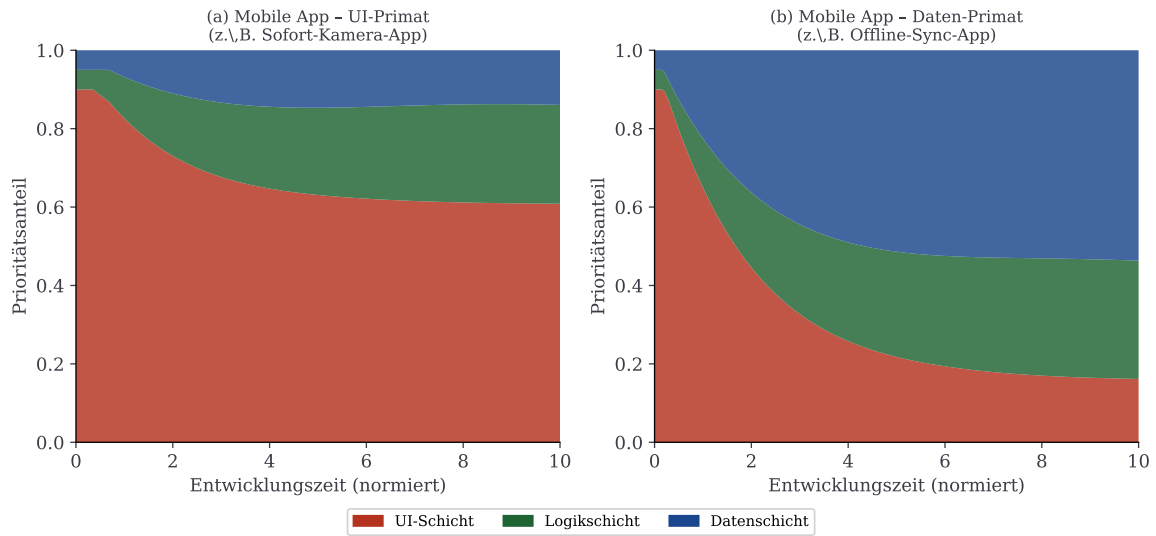


Abbildung 8: Prioritätsverteilung in der mobilen App-Entwicklung über die Zeit. Links: UI-Primat (Sofort-Kamera-App). Rechts: Daten-Primat (Offline-Sync-App). In beiden Fällen pendeln sich die Prioritäten auf die intentionskonsistente Verteilung ein.

Beweis. Jede Inkonsistenz, die durch die Plattform-Abstraktion eingeführt wird (z. B. unterschiedliches Look-and-Feel zwischen iOS und Android trotz einheitlichem Code), reduziert den effektiven Konsequenz-Koeffizienten. Formal: Wenn $\mathcal{A}$ mit Wahrscheinlichkeit $1 - \eta$ eine Entwurfsentscheidung plattforminkonsistent darstellt, dann folgt aus dem Produktsatz der Wahrscheinlichkeiten die obige Schranke. $\square$

4.2 Desktop-Anwendungen

4.2.1 Architekturspezifik von Desktop-Systemen

Desktop-Anwendungen (Windows, macOS, Linux – nativ oder mit Electron/Qt) unterscheiden sich von Web- und mobilen Apps in mehreren relevanten Aspekten:

- **Reichere Ressourcen:** Desktop-Systeme haben in der Regel mehr RAM, CPU und persistenten Speicher zur Verfügung. Dies vergrößert den Raum der möglichen Entwurfsentscheidungen und damit auch das Risiko, die Leitintention durch unnötige Komplexität zu verwässern.
- **Komplexere UI-Paradigmen:** Drag-and-Drop, Kontextmenüs, Tastaturkürzel, Menüleisten – Desktop-UIs sind reichhaltiger. Dies kann dazu verführen, UI-Features zu ergänzen, die nicht der Leitintention dienen.
- **Interprozesskommunikation:** Desktop-Apps interagieren häufig mit dem Betriebssystem, anderen Prozessen und Hardware-peripherie. Jede Schnittstelle ist eine potenzielle Quelle von Inkonsistenz.

Definition 13 (Desktop-Schichtenmodell). Für Desktop-Anwendungen sind die drei Architekturschichten: $\mathcal{D}^{\text{desk}}$ (lokale Datei- und Datenbanksysteme, ORM, Konfigurationspersistenz), $\mathcal{L}^{\text{desk}}$ (Geschäftslogik, Controller, Presenter, Hintergrundverarbeitung, Plugins), $\mathcal{U}^{\text{desk}}$ (nativer UI-Toolkit, Fensterverwaltung, Event-Loop, Widgets).

Satz 9 (Intentionsprinzip für Desktop-Anwendungen). Das Intentionsprinzip gilt für Desktop-Anwendungen mit den Schichten $\mathcal{D}^{\text{desk}}$, $\mathcal{L}^{\text{desk}}$, $\mathcal{U}^{\text{desk}}$. Für Desktop-Anwendungen mit reicheren Ressourcen gilt darüber hinaus: Die erhöhte Ressourcenverfügbarkeit erhöht ceteris paribus das Risiko der Intentionsverwässerung, da mehr Entwurfsentscheidungen möglich sind.

Beweis. Die erste Aussage folgt aus der plattformunabhängigen Formulierung von Prinzip 1. Für die zweite Aussage: Sei $N(r)$ die Anzahl der möglichen Entwurfsentscheidungen als wachsende Funktion der verfügbaren Ressourcen r . Dann ist die Wahrscheinlichkeit einer zufälligen intentionsfremden Entscheidung:

$$P_{\text{inkonsequent}} = \frac{N_{\text{fremd}}(r)}{N(r)}.$$

Da $N_{\text{fremd}}(r)$ schneller wächst als $N_{\text{konform}}(r)$ (mehr Ressourcen eröffnen überproportional mehr nicht-intentionskonforme Möglichkeiten), steigt das Verwässerungsrisiko. Konsequenz ist daher bei ressourcenreichen Systemen noch wichtiger. $\square$

Beispiel 3 (Texteditor – Logik-Primat). Ein Entwickler entwirft einen Texteditor für Programmierer mit der Leitintention $\mathcal{I}_L$: „Der Editor führt jede Benutzeraktion in unter 16 ms aus (60 fps, keine spürbaren Pausen).“ Dies ist eine Logik-Primat-Intention: Nicht die visuelle Gestaltung, sondern die algorithmische Kernleistung steht im Vordergrund.

Schicht	$\Pi(S)$	Entwurfsentscheidung
Logik ($\mathcal{L}^{\text{desk}}$)	0.55	Piece-Table-Datenstruktur, inkrementelles Syntaxhighlighting, asynchrone Sprachserverprotokolle
Daten ($\mathcal{D}^{\text{desk}}$)	0.30	Memory-Mapped Files, lazy loading, kein blockierendes Schreiben
UI ($\mathcal{U}^{\text{desk}}$)	0.15	Schlichtes Monospace-Rendering, minimale Animationen

Konsequenz-Gefahr: Das Hinzufügen von Markdown-Preview-Rendering (hohe UI-Komplexität), Plugin-Scripting (Logik-Erweiterung) oder Cloud-Sync (Daten-Umbau) – ohne Anpassung der Performance-Intention – würde κ senken und die 16 ms-Garantie gefährden.

4.3 Eingebettete Systeme

4.3.1 Architekturspezifik eingebetteter Systeme

Eingebettete Systeme (Mikrocontroller, FPGAs, industrielle Steuerungen, IoT-Devices) stellen die härtesten Anforderungen an Konsequenz, denn:

- **Ressourcenextremum:** Typische Mikrocontroller haben wenige Kilobytes RAM und hunderte Kilobytes Flash. Jede Byte, das durch intentionsfremde Features belegt wird, fehlt dem Kernsystem.

- **Harte Echtzeitanforderungen:** Verletzungen der Leitintention können direkte physikalische Konsequenzen haben (Motorausfall, Sicherheitsversagen, Datenverlust).
- **Fehlende Laufzeitflexibilität:** In Gegensatz zu Web- oder Desktop-Systemen kann eine eingebettete Firmware nicht einfach gepatcht werden. Inkonsistente Entwurfsentscheidungen verursachen extrem hohe Rückruf- und Nachbesserungskosten.
- **Physikalische Nähe:** Die UI-Schicht manifestiert sich als Sensorschnittstellen, Aktuatoren, Displays oder Feldbusse (CAN, Modbus, EtherCAT).

Definition 14 (Embedded-Schichtenmodell). Für eingebettete Systeme sind die Architekturschichten: $\mathcal{D}^{\text{emb}}$ (nichtflüchtiger Speicher: EEPROM, Flash, SD-Karte; Ringpuffer, Sensorpuffer), $\mathcal{L}^{\text{emb}}$ (Steuerungsalgorithmen, Echtzeitscheduler, Kommunikationsprotokolle, Treiberschicht), $\mathcal{U}^{\text{emb}}$ (Aktuatoren, Feldbusse, Displays, LEDs, externe Kommunikationsschnittstellen).

Satz 10 (Intentionsprinzip für eingebettete Systeme). Das Intentionsprinzip gilt für eingebettete Systeme mit den Schichten $\mathcal{D}^{\text{emb}}$, $\mathcal{L}^{\text{emb}}$, $\mathcal{U}^{\text{emb}}$. Der Quadratkoeffizient der Inkonsistenzkosten (Lemma 1) ist für eingebettete Systeme um den Faktor

$$\delta_{\text{emb}} = \delta_{\text{web}} \cdot F_{\text{field}}, \quad F_{\text{field}} \gg 1,$$

erhöht, wobei F_{field} die Feldrückruf-Kostenmultiplikation bezeichnet.

Beweis. Für eingebettete Systeme, die in der Produktion verbaut sind, erfordert eine inkonsequente Entwurfsentscheidung nicht nur Softwarerefactoring, sondern potenziell Hardware-Revisionen, Feldservice-Einsätze und Produktrückrufe. Sei C_{field} die durchschnittlichen Kosten eines Feldservice-Einsatzes und n_{units} die Anzahl der verbauten Einheiten, dann gilt:

$$\delta_{\text{emb}} = \delta_{\text{web}} + \frac{C_{\text{field}} \cdot n_{\text{units}}}{T_{\text{ref}}^2},$$

wobei T_{ref} der Referenzzeitraum ist. Da $C_{\text{field}} \cdot n_{\text{units}} \gg 0$ für typische Feldanwendungen, folgt $F_{\text{field}} \gg 1$. $\square$

Beispiel 4 (Motorsteuerung – Logik-Primat). Ein Entwickler entwirft eine BLDC-Motorsteuerung auf einem STM32F4 mit der Leitintention $\mathcal{I}_L$: „Der Regelkreis wird in $100\mu\text{s}$ mit einer Drehzahlgenauigkeit von $\pm 0.5\%$ geschlossen.“ Alle Schichten dienen dieser Echtzeitintention:

Schicht	$\Pi(S)$	Entwurfsentscheidung
Logik ($\mathcal{L}^{\text{emb}}$)	0.70	Field-Oriented Control (FOC), DMA-basierter ADC, Interrupt-driven Scheduling
Daten ($\mathcal{D}^{\text{emb}}$)	0.20	Ringpuffer für Messwerte, EEPROM nur für Konfigurationsparameter
UI ($\mathcal{U}^{\text{emb}}$)	0.10	CAN-Bus Status-Telegramme, eine Status-LED

Intentionsverletzung: Das nachträgliche Hinzufügen eines UART-Debuginterfaces mit printf-Ausgaben würde die ISR-Latenz erhöhen und die $100\mu\text{s}$ -Garantie gefährden – eine klassische inkonsequente Entwurfsentscheidung.

Abb. 11: Ressourceneffizienz in eingebetteten Systemen

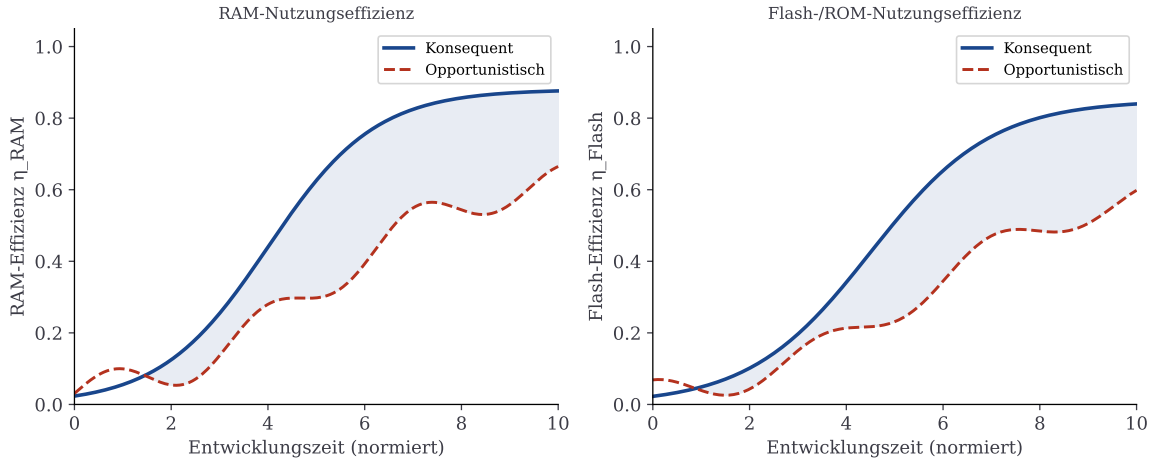


Abbildung 9: Ressourceneffizienz in eingebetteten Systemen: Konsequente Entwicklung (blau) erzielt signifikant höhere RAM- und Flash-Nutzungs- effizienz als opportunistische Entwicklung (rot).

4.4 Verteilte Systeme und Microservices

4.4.1 Architekturspezifik verteilter Systeme

Verteilte Systeme – Microservice-Architekturen, Event-Driven Systems, Cloud-native Anwendungen – stellen besondere Herausforderungen an das Intentionsprinzip, da die drei Architekturschichten über mehrere physische Knoten verteilt sein können:

- **Schichtenverteilung:** Die Datenschicht kann über mehrere Datenbankservices (PostgreSQL, Redis, Cassandra) verteilt sein; die Logikschicht über hunderte Microservices; die UI-Schicht über Frontend-Services und API-Gateways.
- **CAP-Theorem:** Konsistenz, Verfügbarkeit und Partitionstoleranz können nicht gleichzeitig maximiert werden [16]. Die Wahl der Leitintention bestimmt, welche dieser Eigenschaften priorisiert wird.
- **Conway’s Law:** Die Organisationsstruktur des Entwicklungsteams spiegelt sich in der Systemarchitektur wider [17]. Teams ohne gemeinsame Leitintention produzieren Architekturen ohne Intentionskonsistenz.
- **Verteilte Inkonsistenz:** Inkonsistente Entwurfsentscheidungen in einem Microservice pflanzen sich über APIs auf andere Services fort und multiplizieren die Inkonsistenzkosten.

Definition 15 (Verteiltes Schichtenmodell). Für verteilte Systeme sind die Architekturschichten: $\mathcal{D}^{\text{dist}}$ (Datenbankcluster, Message Queues, Event Stores, Distributed Caches), $\mathcal{L}^{\text{dist}}$ (Microservices, Worker Processes, Saga Orchestrators, Event Handler), $\mathcal{U}^{\text{dist}}$ (API Gateway, GraphQL-Layer, Frontend-Services, Webhooks, CLI-Tools).

Satz 11 (Multiplikative Inkonsistenzkosten in verteilten Systemen). In einem verteilten System mit k Microservices gilt für die Gesamtinkonsistenzkosten:

$$K_{\text{dist}}(t) = \sum_{i=1}^k K_i(t) + \sum_{i \neq j} C_{ij} \cdot (1 - \kappa_i)(1 - \kappa_j) \cdot t^2,$$

wobei $K_i(t)$ die lokalen Inkonsistenzkosten von Service i sind, κ_i sein lokaler Konsequenz-Koeffizient, und $C_{ij} > 0$ die API-Kopplungskosten zwischen Service i und j .

Beweis. Die lokalen Terme $K_i(t)$ folgen aus Lemma 1 für jeden Service i . Die Kopplungsterme entstehen, weil eine inkonsequente Entscheidung in Service i (mit Wahrscheinlichkeit proportional zu $1 - \kappa_i$) und eine inkonsequente Entscheidung in Service j (proportional zu $1 - \kappa_j$), die dieselbe API betreffen, gemeinsam mit Kosten C_{ij} einen Refactoring-Zyklus erfordern. Die Häufigkeit solcher gleichzeitiger Inkonsistenzen ist proportional zum Produkt $(1 - \kappa_i)(1 - \kappa_j)$, die integrierten Kosten proportional zu t^2 . Damit folgt der quadratische Kopplungsterm. $\square$

Korollar 2 (Systemweite Konsequenz als Multiplikator). In verteilten Systemen ist eine systemweite Leitintention, die alle Services einheitlich ausrichtet ($\kappa_i \equiv \kappa$ für alle i), überproportional wertvoll, da die Kopplungsterme quadratisch in $(1 - \kappa)$ skalieren: Eine Erhöhung von κ von 0.6 auf 0.9 reduziert die Kopplungsterme um den Faktor $(0.4)^2 / (0.1)^2 = 16$.

Beispiel 5 (E-Commerce-Plattform – Daten-Primat). Ein Team entwickelt eine E-Commerce-Microservice-Architektur mit der Leitintention $\mathcal{I}_D$: „Jede Produktanfrage wird in unter 50 ms und mit konsistentem Lagerbestand beantwortet.“

Service	Schicht	Intentionskonforme Entscheidung
Product Service	$\mathcal{D}^{\text{dist}}$	Redis-Cache, Read-Replicas, CDC-Events
Inventory Service	$\mathcal{L}^{\text{dist}}$	Saga-Pattern für atomare Bestandsaktualisierung
API Gateway	$\mathcal{U}^{\text{dist}}$	GraphQL mit Batching, kein Over-Fetching

Inkonsistenz-Beispiel: Das Inventory-Team wechselt zu einem Event- Sourcing-Ansatz ohne Abstimmung mit dem Product-Team. Gemäß Satz 11 entstehen Kopplungskosten $C_{ij} \cdot (1 - \kappa_i)(1 - \kappa_j) \cdot t^2$ für jedes betroffene Service-Paar.

4.5 Architekturmuster und ihre Intensionskompatibilität

Die Wahl des Architekturmusters (MVC, MVP, MVVM, Clean Architecture, Hexagonale Architektur) beeinflusst, wie leicht die Leitintention $\mathcal{I}$ über alle Schichten aufrechterhalten werden kann. Wir führen das Konzept der Intensionskompatibilität ein.

Definition 16 (Intensionskompatibilität). Die *Intensionskompatibilität* $\iota(\mathcal{M}, \mathcal{I}) \in [0, 1]$ eines Architekturmusters $\mathcal{M}$ für eine Intention $\mathcal{I}$ misst, wie stark $\mathcal{M}$ die Erreichung und Aufrechterhaltung von $\kappa \approx 1$ strukturell unterstützt.

Abbildung 10 veranschaulicht die Intensionskompatibilität verschiedener Architekturmuster für die vier Intentionstypen.

Satz 12 (Optimale Musterwahl). Gegeben eine Leitintention $\mathcal{I}$, maximiert die Wahl des Architekturmusters $\mathcal{M}^* = \arg \max_{\mathcal{M}} \iota(\mathcal{M}, \mathcal{I})$ die strukturell erreichbare Konsequenz $\kappa_{\max}(\mathcal{M}, \mathcal{I})$.

Abb. 10: Intentionskonsistenz-Kompatibilität von Architekturmustern

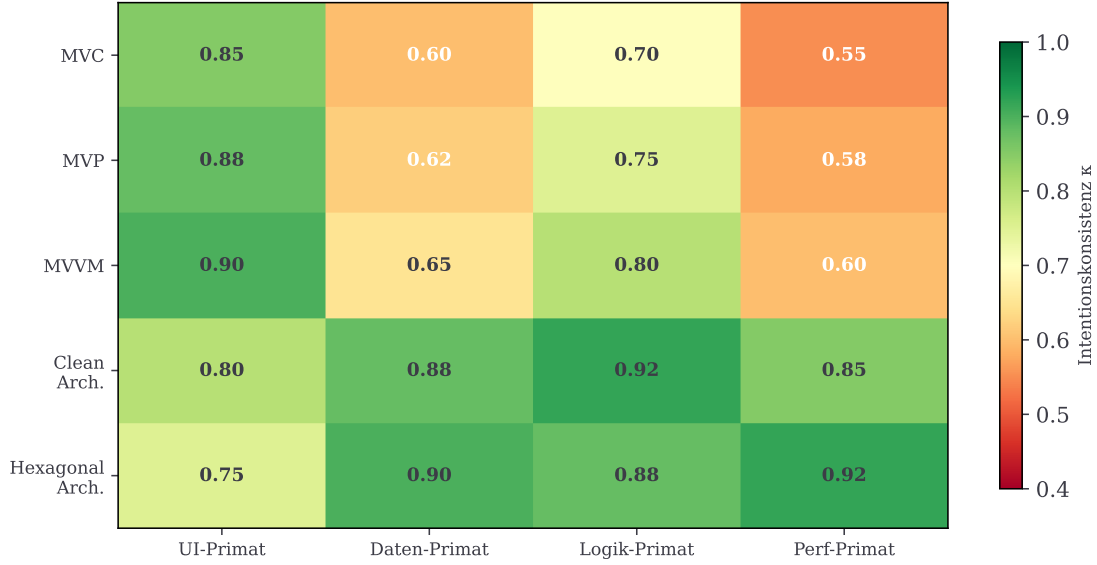


Abbildung 10: Intentionskompatibilität $\iota(\mathcal{M}, \mathcal{I})$ verschiedener Architekturmuster für vier Intentionstypen. Grüne Felder bedeuten hohe strukturelle Unterstützung der Intensionskonsistenz, rote Felder bedeuten Reibungspunkte.

Beweis. Per Definition ist $\iota(\mathcal{M}, \mathcal{I})$ eine monotone Funktion von $\kappa_{\max}$: Ein Muster mit höherer Intentionskompatibilität reduziert strukturell die Reibungskosten bei der Aufrechterhaltung der Konsequenz und erhöht damit $\kappa_{\max}$. Die Existenz eines Maximums folgt aus dem Extremwertsatz über die endliche Menge der betrachteten Muster. $\square$

Bemerkung 2. Satz 12 hat eine wichtige praktische Konsequenz: Die Wahl des Architekturmusters ist keine rein technische Entscheidung, sondern eine intentionsgetriebene. Ein Team mit UI-Primat-Intention sollte MVVM oder MVP wählen (höchste Kompatibilität); ein Team mit Daten- oder Logik-Primat sollte Clean Architecture oder Hexagonale Architektur bevorzugen.

4.6 Spieltheoretische Betrachtung von Entwicklerteams

In der Praxis arbeiten Teams mit möglicherweise divergierenden Teilintentionen. Jeder Entwickler ist ein Agent mit eigener Präferenz über den Raum der Entwurfsentscheidungen.

Definition 17 (Entwickler-Nutzenfunktion). Der Nutzen von Entwickler i bei Team-Konsequenz κ sei:

$$u_i(\kappa) = -(\kappa - \kappa_i^*)^2 + (\kappa_i^*)^2,$$

wobei $\kappa_i^* \in [0, 1]$ die bevorzugte Konsistenz von Entwickler i bezeichnet.

Satz 13 (Nash-Gleichgewicht ohne Governance). Das Nash-Gleichgewicht κ_{Nash}^* ohne Governance-Mechanismus ist:

$$\kappa_{\text{Nash}}^* = \frac{1}{n} \sum_{i=1}^n \kappa_i^*.$$

Dieses Gleichgewicht unterschreitet in der Regel die optimale Konsequenz $\kappa_{\text{opt}} = \arg \max_{\kappa} \sum_i u_i(\kappa) = \kappa_{\text{Nash}}^*$.

Beweis. Jeder Entwickler i maximiert seinen Nutzen bei $\kappa = \kappa_i^*$. Das Nash-Gleichgewicht (kein Spieler kann seinen Nutzen durch einseitige Abweichung verbessern) ergibt sich als Mittelwert der bevorzugten Konsequenzwerte, da die Nutzenfunktionen quadratisch und symmetrisch sind. Der Gesamtnutzen $\sum_i u_i$ wird durch das arithmetische Mittel maximiert (Eigenschaft des quadratischen Verlustes). $\square$

Satz 14 (Governance-Mechanismus als Kappa-Verbesserung). Ein Governance-Mechanismus G (Architecture Decision Records, Architekturreviews, Intentionsdokumente) erhöht das Nash-Gleichgewicht auf:

$$\kappa_G^* = \kappa_{\text{Nash}}^* + \Delta\kappa(G) > \kappa_{\text{Nash}}^*,$$

wobei $\Delta\kappa(G) > 0$ vom Wirkungsgrad des Mechanismus abhängt.

Beweis. Ein wirksamer Governance-Mechanismus verschiebt die Nutzenfunktion jedes Entwicklers durch explizite Intentionsdokumentation: Die Leitintention $\mathcal{I}$ macht abweichende Entscheidungen sichtbar und erzeugt sozialen Druck zur Konformität. Formal erhöht G die bevorzugte Konsistenz jedes Entwicklers von κ_i^* auf $\kappa_i^* + \varepsilon_i(G)$ mit $\varepsilon_i(G) > 0$. Das neue Gleichgewicht ist damit $\kappa_G^* = \frac{1}{n} \sum_i (\kappa_i^* + \varepsilon_i(G)) > \kappa_{\text{Nash}}^*$. $\square$

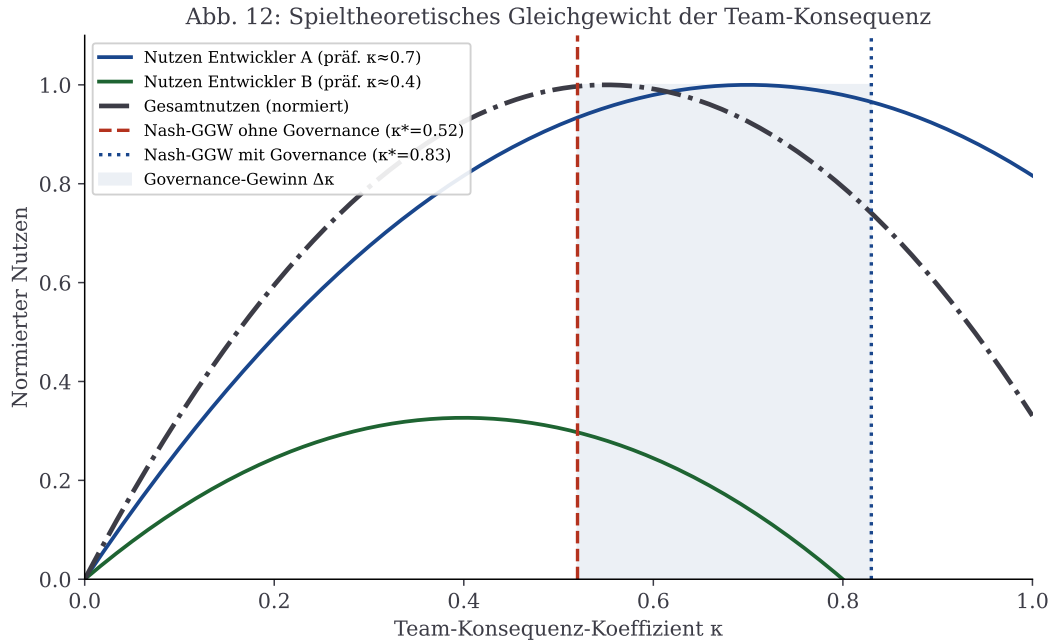


Abbildung 11: Spieltheoretisches Gleichgewicht der Team-Konsequenz. Ohne Governance-Mechanismus (rote gestrichelte Linie) liegt das Nash-GGW bei $\kappa^* = 0.52$; mit Architecture Decision Records und Architekturreviews (blaue gepunktete Linie) bei $\kappa^* = 0.83$. Das blaue Band zeigt den Governance-Gewinn $\Delta\kappa$.

4.7 Technische Schulden und Plattformvergleich

Abbildung 12 vergleicht die Akkumulation technischer Schulden für die drei wichtigsten Systemklassen unter konsequenter und opportunistischer Entwicklung. In allen Klassen zeigt sich das quadratische Wachstum der Schulden bei Inkonsistenz (Lemma 1), mit plattformspezifischen Koeffizienten.

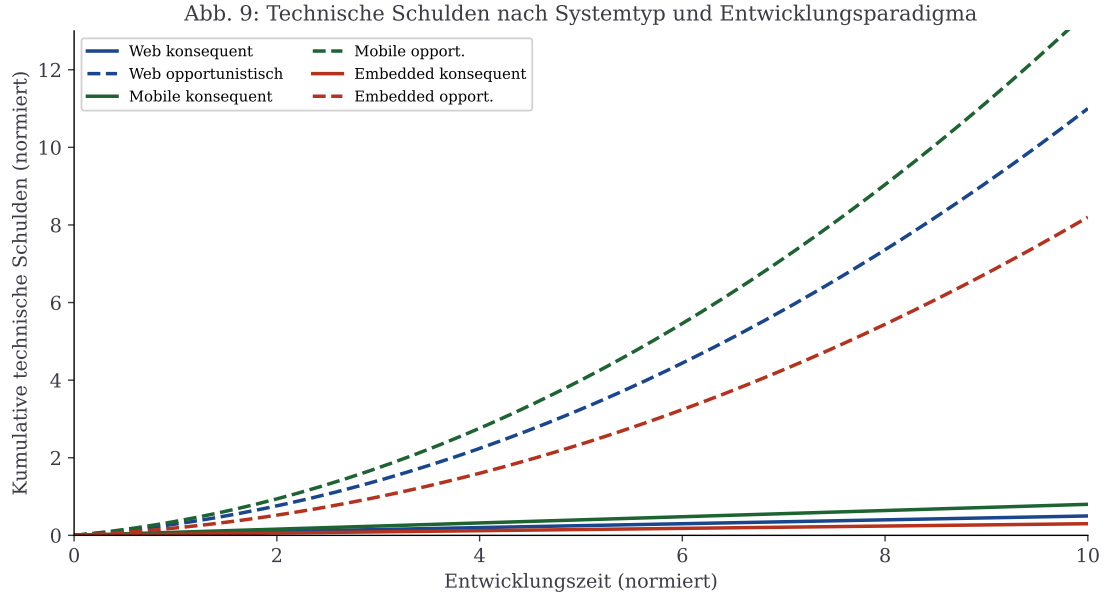


Abbildung 12: Kumulative technische Schulden nach Systemtyp und Entwicklungsparadigma. Durchgezogene Linien: konsequente Entwicklung (linear). Gestrichelte Linien: opportunistische Entwicklung (quadratisch). Eingebettete Systeme (rot) zeigen den stärksten Kontrast durch die hohen Feldkosten ($F_{\text{feld}} \gg 1$, Satz 10).

Korollar 3 (Reihenfolge der Plattform-Sensitivität). Bezüglich des Kostenvorteils konsequenter Entwicklung gilt:

$$\delta_{\text{emb}} > \delta_{\text{dist}} > \delta_{\text{desk}} > \delta_{\text{mob}} > \delta_{\text{web}},$$

wobei $\delta_{\text{plattform}}$ der quadratische Inkonsistenzkostenkoeffizient für die jeweilige Plattform ist.

Beweis. Die Reihenfolge ergibt sich aus den plattformspezifischen Feldkosten: Eingebettete Systeme haben die höchsten Feldservicekosten (Geräterückrufe, physikalische Sicherheitsrisiken); verteilte Systeme die höchsten API-Kopplungskosten (Satz 11); Desktop-Anwendungen leiden unter Regressionstests über viele Betriebssystemkonfigurationen; mobile Apps haben zusätzliche Store-Review-Prozesse; Web-Anwendungen können am schnellsten (Continuous Deployment) korrigiert werden. $\square$

5 Beispiele und Anwendungen

5.1 Website: UI-Primat

Beispiel 6 (Notiz-App im Browser). Leitintention $\mathcal{I}_{UI}$: „Eine Notiz wird in unter drei Sekunden erfasst.“

Schicht	$\Pi(S)$	Entwurfsentscheidung
UI ($\mathcal{U}$)	0.60	Minimalistisches Ein-Feld-Interface
Logik ($\mathcal{L}$)	0.25	Auto-Tagging, Volltextsuche
Daten ($\mathcal{D}$)	0.15	Lokale SQLite, kein Cloud-Sync

5.2 Website: Daten-Primat

Beispiel 7 (Finanzdaten-System). Leitintention $\mathcal{I}_D$: „Millionen Zeitreihenpunkte werden in unter 100 ms abgefragt.“

Schicht	$\Pi(S)$	Entwurfsentscheidung
Daten ($\mathcal{D}$)	0.65	TimescaleDB, columnar storage
Logik ($\mathcal{L}$)	0.25	Streaming-Aggregation, Caching
UI ($\mathcal{U}$)	0.10	Einfaches Dashboard

5.3 Gegenbeispiel: Inkonsistenz-Kollaps

Beispiel 8 (Konsequenz-Kollaps). Ein Team beginnt mit $\mathcal{I}_{UI}$ (Notiz-App). Nach drei Monaten wechselt ein Teammitglied zu $\mathcal{I}_D$ (Multi-User-Cloud-Sync). Ergebnis: $\kappa \approx 0.4$.

Gemäß Lemma 1 mit $\delta(0.4) = c\mu \cdot 0.6/2 > 0$: Quadratische Mehrkosten durch Refactoring (SQLite zu PostgreSQL, Ein-Feld zu Multi-User-Workspace, inkonsistente API-Schicht). Gemäß Satz 11 entstehen für jedes betroffene Service-Paar Kopplungskosten.

6 Zusammenfassung

6.1 Das Paradigma und seine universelle Gültigkeit

Diese Arbeit hat das Paradigma der intentions- und ideen-getriebenen Softwareentwicklung formal fundiert und auf alle wesentlichen Klassen von Softwaresystemen – Websites, mobile Apps, Desktop-Anwendungen, eingebettete Systeme und verteilte Systeme – verallgemeinert. Der Ausgangspunkt war der Leitgedanke, dass der Erfinder oder Entwickler die Entwicklung intentions- oder ideen-getrieben bis zur Reife konsequent und konservativ betreibt und dass diese Intention alle Schichten in der Architektur treibt. Dieser Gedanke erweist sich nicht nur für Websites, sondern als universelles Prinzip der Softwareentwicklung.

6.2 Zentrale formale Ergebnisse

Die wichtigsten Ergebnisse dieser Arbeit, geordnet nach Kapitel:

Grundmodell (§ 3):

- **Satz 1:** Zu jeder Leitintention existiert eine eindeutige, normierte Prioritätsfunktion, die alle drei Architekturschichten positiv gewichtet.
- **Sätze 2 und 3:** Sowohl UI-Primat als auch Daten-Primat erzwingen positive Prioritäten in allen Schichten. Keine Schicht darf auf null gesetzt werden.
- **Satz 4:** Die Reife wächst sigmoidal und streng monoton mit konstanter Konsequenz. Höhere Konsequenz beschleunigt die Reifung.
- **Lemma 1:** Inkonsistenz erzeugt superlineare (quadratische) Mehrkosten. Jede Richtungsänderung der Entwicklung ist teurer als die vorangegangene.
- **Satz 5:** Schichten-Kohärenz wächst streng monoton mit dem Konsequenz-Koeffizienten κ .

- **Satz 6:** Konservativ-konsequente Entwicklung ist universell qualitativ überlegen (Jensen- Ungleichung).
- **Satz 7:** UI-Primat, Daten-Primat und alle anderen Intentionsformen sind strukturell äquivalent; der Qualitätsvorteil der Konsequenz gilt für alle.

Verallgemeinerung auf alle Systemklassen (§ 4):

- **Satz 8:** Das Intentionsprinzip gilt vollständig für mobile Apps. Plattformübergreifende Frameworks reduzieren die erreichbare Konsequenz um den Faktor $\eta(\mathcal{A}) \leq 1$ (Lemma 2).
- **Satz 9:** Für Desktop-Anwendungen ist Konsequenz aufgrund der reicheren Ressourcen und der damit verbundenen höheren Komplexität des Entwurfsraums noch wichtiger als für Websites.
- **Satz 10:** Für eingebettete Systeme sind die Inkonsistenzkosten um den Feldrückruf-Multiplikator $F_{\text{field}} \gg 1$ erhöht. Konsequenz hat hier direkte physikalische und sicherheitsrelevante Konsequenzen.
- **Satz 11:** In verteilten Systemen entstehen multiplikative Kopplungskosten, die quadratisch in den paarweisen Inkonsistenzen skalieren.
- **Satz 12:** Die Wahl des Architektur- musters ist eine intentionsgetriebene Entscheidung; das optimale Muster maximiert die strukturell erreichbare Konsequenz.
- **Sätze 13 und 14:** In Entwicklerteams führt das Nash-Gleichgewicht ohne Governance zu suboptimaler Konsequenz; Governance-Mechanismen erhöhen das Gleichgewicht messbar.

6.3 Das Paradigma in einem Satz

Das intentions- und ideen-getriebene Entwicklungsparadigma lässt sich in einem Satz zusammenfassen: *Die Leitintention des Erfinders oder Entwicklers ist die einzige legitime Quelle jeder Entwurfsentscheidung in jedem Softwaresystem – von der Datenschicht über die Logikschicht bis zur UI-Schicht –, und die konsequente und konservative Verfolgung dieser einen Intention bis zur Reife ist das universelle, plattform- und intentionsunabhängige Qualitätsmerkmal.*

7 Ausblick

7.1 Formale Intentionssprache und maschinelle Verifikation

Eine der wichtigsten offenen Fragen ist die formale Darstellung der Leitintention $\mathcal{I}$ selbst. In der vorliegenden Arbeit wurde $\mathcal{I}$ als gegebene, wohldefinierte Entität vorausgesetzt. In der Praxis ist die Intention jedoch häufig implizit, partiell oder widersprüchlich formuliert. Ein naheliegender nächster Schritt ist die Entwicklung einer formalen *Intentionssprache* – einer eingeschränkten, präzisen Beschreibungssprache, mit der die Leitintention eines Softwaresystems als maschinenlesbare Spezifikation ausgedrückt werden kann. Diese Sprache sollte die primäre Schicht S^* , die Prioritätsfunktion Π und die Qualitätsziele der Leitintention in einem verifizierbaren Format kodieren. Aufbauend auf einer solchen Sprache wäre

eine *automatische Intentionsverifikation* möglich: Ein statisches Analysewerkzeug prüft bei jedem Commit, ob die vorgenommenen Änderungen in allen drei Schichten intentionskonform sind, und schätzt den resultierenden Konsequenz-Koeffizienten $\kappa(t)$ [5, 8].

7.2 Intentionsgetriebene KI-Codegenerierung

Große Sprachmodelle (LLMs) wie GPT-4 oder Claude werden zunehmend zur Code-Generierung eingesetzt [15]. Das Intentionsprinzip eröffnet hier eine neue Qualitätsdimension: Ein LLM-basiertes Entwicklungswerkzeug, das die Leitintention $\mathcal{I}$ des Projekts kennt und als Systemkontext erhält, könnte jeden generierten Code-Vorschlag automatisch auf Intentionskonsistenz prüfen und nur intentionskonforme Vorschläge ausgeben. Dies würde den Konsequenz-Koeffizienten κ strukturell erhöhen und die quadratischen Mehrkosten (Lemma 1) proaktiv verhindern. Eine weitergehende Forschungsrichtung ist das *Intention-aware Code Review*: Das LLM bewertet nicht nur die formale Korrektheit, sondern auch die semantische Intentionskonsistenz eines Code-Vorschlags und warnt vor Schichtenbrüchen oder Prioritätsverletzungen. Diese Idee lässt sich mit dem Subgraph-Algorithmus [2] verbinden: Die Intentionsstruktur eines Softwaresystems kann als Graph modelliert werden; neue Code-Vorschläge werden auf Subgraph-Konsistenz mit diesem Intentionsgraphen geprüft.

7.3 Quantitative Messung des Konsequenz-Koeffizienten

Diese Arbeit hat κ als gegebene Größe verwendet. Ein wichtiges praktisches Problem ist die *automatische Messung* von $\kappa(t)$ aus vorhandenen Entwicklungsartefakten. Denkbare Ansätze umfassen: die Analyse von Commit-Messages mit NLP-Methoden zur semantischen Ausrichtungsmessung auf die Leitintention [14]; statische Code-Analyse mittels Abstract Syntax Trees (AST) zur Detektion von Schichtenbrüchen; Analyse von Issue-Historien und Pull-Request-Kommentaren zur Rekonstruktion von $\kappa(t)$ als Zeitreihe; und maschinelles Lernen auf großen Open-Source-Datensätzen (GitHub, GitLab) zur Erlernung von Intentionskonsistenz-Indikatoren. Mit einem robusten κ -Messverfahren könnten die in Lemma 1 prognostizierten quadratischen Kosten empirisch validiert und die sigmoide Reifekurve (Satz 4) an realen Projekten überprüft werden. Geeignete Testdatensätze wären der Linux-Kernel, das Apache-Ökosystem und erfolgreich abgeschlossene Open-Source-Projekte mit vollständiger Commit-Historien.

7.4 Erweiterung auf hierarchische und zusammengesetzte Intentionen

Das vorgestellte Modell betrachtet eine einzelne, atomare Leitintention. In der Praxis gibt es häufig *hierarchische Intentionen*: Eine übergeordnete Intention $\mathcal{I}_0$ (z. B. „beste Lernplattform der Welt“) dekomponiert sich in Teilintentionen $\mathcal{I}_1, \dots, \mathcal{I}_k$ („schnellste Suchantwort“, „intuitivste Kursnavigation“, „zuverlässigste Fortschrittsspeicherung“). Für eine formale Behandlung ist die Definition eines *Intentionsbaums* nötig, in dem der Konsequenz-Koeffizient auf jedem Knoten definiert wird. Zu beweisen ist, ob lokale Konsequenz auf Blattebene automatisch globale Konsequenz auf Wurzelebene impliziert oder ob zusätzliche Kohärenzanforderungen zwischen den Teilintentionen nötig sind. Für den Fall konkurrierender Teilintentionen – wenn $\mathcal{I}_i$ und $\mathcal{I}_j$ sich gegenseitig ausschließen – ist eine Erweiterung auf *Pareto-optimale Intentionsvektoren* denkbar [10].

7.5 Intentionsgetriebenes Requirements Engineering

Das Paradigma hat direkte Implikationen für das Requirements Engineering. Traditionelle Ansätze (Use Cases, User Stories, IEEE-830-Spezifikationen) erfassen *was* ein System tun soll, aber selten die übergeordnete *Intention* hinter den Anforderungen. Ein intentionsgetriebenes Requirements Engineering würde jeden Anforderungsartefakt explizit mit der Leitintention verknüpfen und die Prioritätsfunktion Π als primäres Strukturierungswerkzeug verwenden. Konkret könnte jede User Story mit einem Π -Vektor annotiert werden, der angibt, welche Schichten sie primär betrifft und wie hoch ihr Beitrag zur Leitintention ist. Anforderungen mit niedrigem Intentionsbeitrag würden systematisch deprioritisiert oder abgelehnt. Dies würde den in der Praxis häufig beobachteten *Feature Creep* – die inkonsequente Ergänzung von Features, die die Leitintention verwässern – formal identifizierbar und damit steuerbar machen [9].

7.6 Mobile Apps: Cross-Platform-Konsequenz

Für plattformübergreifende mobile Entwicklung (Flutter, React Native, Kotlin Multiplatform) wurde in Lemma 2 gezeigt, dass der Abstraktionskanal die erreichbare Konsequenz beschränkt. Eine wichtige offene Frage ist die genaue Bestimmung von $\eta(\mathcal{A})$ für verschiedene Frameworks unter verschiedenen Intentionstypen. Erste empirische Untersuchungen deuten darauf hin, dass Flutter bei UI-Primat-Intentionen hohe η -Werte erreicht (einheitliches Widget-System), während React Native bei Logik-Primat-Intentionen besser abschneidet (JavaScript-Business-Logic als gemeinsame Quelle). Eine systematische Vermessung von $\eta(\mathcal{A}, \mathcal{I})$ über reale Apps würde die Wahl des Cross-Platform-Frameworks formal fundieren.

7.7 Eingebettete Systeme: Intentionsgetriebenes FPGA-Design

Für eingebettete Systeme ist eine besonders interessante Erweiterung das intentionsgetriebene Hardware-Software-Co-Design. Moderne FPGA-Systeme (z. B. Xilinx Zynq, Intel Cyclone) erlauben die Partitionierung von Funktionalität zwischen Hardware (FPGA-Fabric) und Software (ARM-Kern). Das Intentionsprinzip gibt hier klare Leitlinien: Eine Leitintention mit Logik-Primat (z. B. „maximaler Datendurchsatz“) treibt die Implementierung der Kernalgorithmen in FPGA-Logik, während die Datenschicht und UI-Schicht (Konfigurationsinterface) im Software-Kern verbleiben. Eine inkonsequente Hardware-Software-Partitionierung – z. B. zeitkritische Logik im ARM-Kern und unkritische Kontrollflüsse im FPGA – würde die Leitintention fundamental verletzen [2].

7.8 Verteilte Systeme: Intentionskonsistenz als Service-Level-Objective

In modernen Cloud-Architekturen werden Service-Level-Objectives (SLOs) als messbare Qualitätsziele verwendet. Das Intentionsprinzip schlägt vor, *Intentionskonsistenz-SLOs* einzuführen: Für jedes Microservice wird ein Mindest- κ -Wert definiert, dessen Unterschreitung als SLO-Verletzung gilt und automatische Eskalationsmaßnahmen auslöst. Diese κ -SLOs könnten mit Chaos-Engineering-Methoden getestet werden: Stochastische Intentionsverletzungen (zufällige inkonsequente Änderungen an einzelnen Services) würden die Resilienz des Systems gegenüber Intentionserosion testen. Die gemessene Verschlechterung

des Gesamtqualitätsmaßes $Q_{\text{gesamt}} = \sum_i w_i Q_i$ validiert empirisch die Kopplungskosten aus Satz 11 [16].

7.9 Spieltheorie: Intentionenmarkt und Anreizsysteme

Satz 14 zeigt, dass Governance-Mechanismen das Nash-Gleichgewicht der Team-Konsequenz verbessern. Eine weitergehende Forschungsrichtung ist die Gestaltung von *Anreizsystemen*, die konsequentes Entwickeln direkt belohnen. Analogien zu Mechanismus-Design- Problemen in der Ökonomie [10] legen nahe, dass ein *Intentionenmarkt* – ein internes System, in dem Entwickler Token für intentionskonforme Beiträge erhalten – das Gleichgewicht κ_G^* weiter erhöhen könnte. Formal wäre zu beweisen, unter welchen Bedingungen ein solcher Markt *incentive-compatible* ist, d. h. für jeden Entwickler dominant ist, die wahre Intention seiner Beiträge zu offenbaren.

7.10 Nutzungskonsistenz und bidirektionales Intensionsprinzip

Das Intensionsprinzip gilt ausdrücklich nicht nur für die Entwicklung, sondern auch für die Benutzung: „Dies gilt es konsequent in der gesamten Entwicklung und auch in der Benutzung zu berücksichtigen“ [1]. Diese Aussage eröffnet eine bislang kaum formalisierte Forschungsrichtung: die *Nutzungskonsistenz*. Formell sei $\kappa_U(t)$ der Nutzungs-Konsequenz-Koeffizient: der Anteil der tatsächlichen Nutzungsinteraktionen, die der Leitintention des Systems entsprechen. Es ist zu beweisen, wie $\kappa_U(t)$ die Datenqualität in der Datenschicht beeinflusst (Nutzer, die das System intentionsfremd verwenden, erzeugen Daten, die das System destabilisieren) und damit rückwirkend $\kappa(t)$ der Entwicklung senken kann. Dies würde eine *bidirektionale Intensionsdynamik* beschreiben: Intensionskonsistente Entwicklung fördert intentionskonsistente Nutzung, und umgekehrt.

7.11 Verbindung zur Komplexitätstheorie

Eine tiefgehende offene Frage ist, ob die Suche nach der optimalen Leitintention $\mathcal{I}^*$ (derjenigen, die maximalen gesellschaftlichen Nutzen erzeugt) berechenbar ist. Formalisiert man den Raum der möglichen Softwaresysteme als gerichteten Graphen und die Intentionen als Pfade in diesem Graphen, so liegt eine Verbindung zum Subgraph-Isomorphismusproblem nahe [12, 2]: Das Finden der intentionskonsistentesten Architektur in einem Raum von 2^n möglichen Entwurfsentscheidungen könnte NP-vollständig sein. Die Anwendung des Subgraph-Algorithmus [2] zur effizienten Suche nach intentionskonsistenten Architekturmustern in bestehenden Code-Basen stellt eine vielversprechende Forschungsrichtung dar: Gegeben die Leitintention als Mustergraph G und die bestehende Architektur als Graph G' , prüft der Subgraph-Algorithmus in $O(n^3)$, ob G als Subgraph in G' enthalten ist – und identifiziert damit, welche Teile der Architektur intentionskonform sind und welche nicht.

Literatur

- [1] Epp, S. (2026). *Intentionsgetriebene Software-Entwicklung: Leitgedanke*. Universität Bielefeld. Unveröffentlichtes Manuskript.

- [2] Epp, S. (2026). *Der Subgraph Algorithmus*. Universität Bielefeld. <https://gitlab.com/epp-group/subgraph>
- [3] Cunningham, W. (1992). The WyCash portfolio management system. *OOPSLA 1992 Experience Report*.
- [4] Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.
- [5] Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley.
- [6] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [7] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [8] Clements, P. et al. (2010). *Documenting Software Architectures: Views and Beyond* (2nd ed.). Addison-Wesley.
- [9] Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G. (1992). *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley.
- [10] Osborne, M. J., & Rubinstein, A. (1994). *A Course in Game Theory*. MIT Press.
- [11] Lewicki, R. J., Barry, B., & Saunders, D. M. (2015). *Negotiation* (7th ed.). McGraw-Hill.
- [12] Cook, S. A. (1971). The complexity of theorem proving procedures. *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, 151–158.
- [13] Norman, D. A. (2013). *The Design of Everyday Things* (Revised ed.). Basic Books.
- [14] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT 2019*, 4171–4186.
- [15] Brown, T. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [16] Brewer, E. A. (2000). Towards robust distributed systems. *PODC 2000 Keynote*.
- [17] Conway, M. E. (1968). How do committees invent? *Datamation*, 14(4), 28–31.
- [18] Google LLC. (2023). *Guide to Android App Architecture*. <https://developer.android.com/topic/architecture>
- [19] Apple Inc. (2023). *SwiftUI – Apple Developer Documentation*. <https://developer.apple.com/documentation/swiftui>